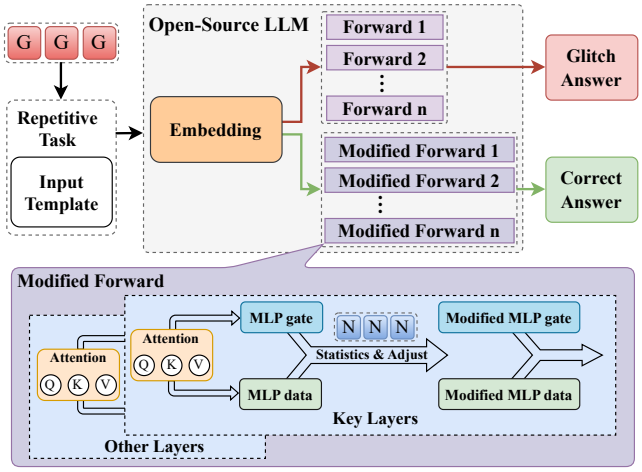

Figure 5: GlitchProber workflow for fixing glitch tokens.

Initially, the algorithm samples a subset of tokens (S) from the vocabulary (V) based on a predefined sampling rate (γ) (line 1). For each token in this subset, the algorithm extracts relevant features using a transformer model over specified key layers and assigns labels indicating whether each token is glitchy (lines 2-5).

Following the feature extraction, the algorithm applies PCA to reduce the dimension of these features to a lower-dimensional space (P) (line 6). The reduced feature set (F) is then used to train a SVM classifier with the assigned glitch labels (line 7).

For tokens not included in the initial sample, the algorithm uses the trained SVM classifier to predict whether each token is normal or a glitch (lines 8-12). Tokens classified as glitches will undergo a further validation step to confirm their status, effectively minimizing the false positive rate (lines 14-18). Finally, the algorithm compiles and outputs two sets, namely glitch token set (G) and normal token set (N).

4.2 Fixing Glitch Tokens via GlitchProber

We further explore the possibility of correcting the anomalous activation patterns of glitch tokens to normal patterns by adjusting the activation values of the model's intermediate layers, thereby eliminating their negative impact on the model output. Based on the idea of adjusting neuron activation values to eliminate the influence of glitch tokens, we focus on two types of neurons in normal tokens, i.e., the neurons that are activated in the vast majority of normal tokens, and the neurons that are not activated in any normal tokens. Then we compare the differences in activation values of these key neurons between normal tokens and glitch tokens. By simulating those normal activation patterns, we achieve adaptively adjustment of the activation values of glitch tokens. The overall approach is illustrated in Figure 5.

4.2.1 ***Normal Token Activation Value Statistics.*** GlitchProber first randomly samples a subset of normal tokens, i.e., $N' \subset N$, to calculate the activation value distribution of normal tokens in the target layers. We set the sampling rate to γ, which is consistent with the sampling rate in the glitch token detection phase.

For the MLP module in each layer, we calculate the activation statistics of the tokens in the normal token set N'. We define two sets of neuron indices as $Neun^{\uparrow}$ and $Neun^{\downarrow}$ based on their activation patterns, where $Act[i]$ represents the activation value of the i-th neuron in the MLP module, and m is the predefined threshold.

$$Neun^{\uparrow} = \{i \,|\, Act[i] > m \text{ for over 99\% of tokens in } N'\} \quad (7)$$

$$Neun^{\downarrow} = \{i \,|\, Act[i] \leq m \text{ for all tokens in } N'\} \quad (8)$$

$Neun^{\uparrow}$ denotes the set of key neurons that exhibit high activation levels, surpassing predefined threshold m, across sample token set N'. Given that activated neurons constitute a small proportion of the total neurons, we consider a neuron to be a key neuron if it is activated in over 99% of the tokens. Conversely, $Neun^{\downarrow}$ represents the set of key neurons that exhibit consistently low activation levels, falling below m. We consider a neuron to be a key neuron in $Neun^{\downarrow}$ if its activation level remains below the threshold m for all tokens in N'. These neurons can be considered as the key features for suppressing noise, as they are consistently inactive for normal tokens. By identifying these two sets of neuron indices based on their activation patterns, we can create a profile of the expected behavior of normal tokens at each MLP module. Those methods ensure the recorded neurons take the most informative features when we mitigate the influence of noise and irrelevant information caused by glitch tokens.

Note that we only adjust the activation values of the MLP module and not the attention patterns. Attention patterns capture the relative importance between tokens, and modifying them may disrupt token dependencies and introduce noise. In contrast, MLP activation values reflect the model's understanding of each token independently, and adjusting these values has less impact on token relationships. By only adjusting MLP activation values, we aim to fix glitch tokens without introducing additional noise.

4.2.2 ***Activation Value Adjustment.*** When the hooked model processes detected glitch tokens, GlitchProber intervenes in this process, making trend-based adjustments to the neurons identified by $Neun^{\uparrow}$ and $Neun^{\downarrow}$. For neurons in $Neun^{\uparrow}$ that should be activated but have insufficient activation in glitch tokens, the algorithm uses β as an amplification factor to promote their activation. Conversely, for neurons in $Neun^{\downarrow}$ that should be suppressed but are abnormally activated in glitch tokens, the algorithm uses α as a reduction factor to suppress their anomalous activation. The factors β and α are named adjustment factors, which play a crucial role in the glitch token fixing process. For the calculation of β and α, we first calculate the average activation value difference $\Delta Act^{\uparrow}$ of glitch tokens relative to normal tokens on highly activated neurons ($Neun^{\uparrow}$), and the average activation value ratio $\Delta Act^{\downarrow}$ on lowly activated neurons ($Neun^{\downarrow}$).

$$\Delta Act^{\uparrow} = \frac{1}{|Neun^{\uparrow}|} \sum_{i \in Neun^{\uparrow}} (Act_{\text{normal}}[i] - Act_{\text{glitch}}[i]) \quad (9)$$

$$\Delta Act^{\downarrow} = \frac{1}{|Neun^{\downarrow}|} \sum_{i \in Neun^{\downarrow}} \left(\frac{Act_{\text{glitch}}[i]}{Act_{\text{normal}}[i]} \right) \quad (10)$$

Subsequently, through linear transformation and range restriction, the algorithm maps $\Delta Act^{\uparrow}$ and $\Delta Act^{\downarrow}$ to appropriate numerical intervals to obtain the values of β and α, respectively.

$$\beta = k_1 \cdot \Delta Act^{\uparrow} + b_1 \tag{11}$$

$$\alpha = k_2 \cdot \Delta Act^{\downarrow} + b_2 \tag{12}$$

The constants k_1, b_1, k_2, and b_2 are derived through an adaptive process tailored to the specific dynamics of each model. A set of default values is provided, which can be adjusted based on empirical data to optimize the correction process for different types of models. They are crucial for ensuring β and α effectively modulate neuron activations while maintaining system stability and performance.

After the adjustment of the MLP activation values is completed, we input the corrected activation values back into the subsequent layers, allowing the model to continue the forward propagation until the final fixed result is output. This process corrects on each token in the detected glitch token set G.

Algorithm 2: GlitchProber (Fix)

```
Input: Glitch token set G; Normal token set N; Sample Rate γ;
       Threshold m; KeyLayers[]
1  N' ← randomSample(N, γ);
2  Neun↑, Neun↓ ← statisticsNeuron(N', m);
3  β ← statisticsBeta(N', Neun↑);
4  α ← statisticsAlpha(N', Neun↓);
5  foreach Token ∈ G do
6      for Layer ∈ KeyLayers do
7          Activation ← hookModel(Token, Layer);
8          foreach Neuron ∈ Neun↑ do
9              Act[Neuron] ← Act[Neuron] + β;
10         end
11         foreach Neuron ∈ Neun↓ do
12             Act[Neuron] ← Act[Neuron]/α;
13         end
14         hookModel(Token, Layer) ← Activation;
15     end
16 end
```

4.2.3 ***Fixing Process Algorithm.*** Based on Section 4.2.2, we present the pseudocode for the fix algorithm of GlitchProber in Algorithm 2. The algorithm begins by sampling a subset (N') of normal tokens from the full set of normal tokens (N) (line 1). Then it computes the statistical distribution of activation values across key neurons in the subset, distinguishing $Neun^{\uparrow}$ and $Neun^{\downarrow}$ (line 2). After that the algorithm calculates the adjustment factors β and α for the identified key neurons (line 3-4).

For each glitch token in the set G, the algorithm iteratively applies these adjustments across specified layers (i.e., KeyLayers) of the model (lines 5-7). Neurons in $Neun^{\uparrow}$ have their activation values increased by β, amplifying their response to mimic normal activation patterns (lines 8-10). Conversely, neurons in $Neun^{\downarrow}$ have their activation values reduced by dividing by α, suppressing any abnormal activations (lines 11-13). Each adjusted activation is reintegrated into the model's processing flow, allowing it to continue with forward propagation with the corrected values (line 14).

4.3 Key Layers Selection

In the design of our GlitchProber's detection and fix algorithms, we focused on exploiting the attention pattern and MLP status features within certain *key layers*. The rationale behind selecting these key layers stems primarily from our empirical findings outlined in Finding 2 (Section 3.2), which highlighted that glitch tokens predominantly affect the model's downstream layers closer to the output. For instance, in Llama2, this pertains to layers 19 to 31. Further refining our selection, we encountered a counter-intuitive discovery: modifying features in layers exceedingly close to the output paradoxically diminished the effectiveness of our fix algorithms.

The layers preceding the final output are crucial for tailoring responses based on preceding computations; alterations in these layers can disrupt representational balances, leading to degraded performance. In our approach for Llama2, we designated layers 19 to 28 as key layers, optimally positioned in the middle to lower sections of the model's architecture. This strategic placement ensures that our interventions effectively mitigate the effects of glitch tokens while preserving the model's robustness.

5 EVALUATION OF GLITCH TOKEN DETECTION

5.1 Experiment setup

Experiment Environment. All experiments are performed on a workstation with Ubuntu 22.04.3 LTS and 250GB memory, and 2 A100 GPU with 80GB memory each.

LLM Selection. We thoroughly evaluated our proposed method using a diverse set of computational models. We selected five widely recognized, open-source models, including Llama-2-7b-chat [36], Mistral-7B-Instruct-v0.1 [18], Qwen-7B-Chat [5], Gemma-2b-it [14], and Yi-6B-Chat [3]. These models served as the subjects for our in-depth analysis, allowing us to assess the versatility and effectiveness of our method across various real-world applications. Table 1 provides an overview of these models' parameters.

Evaluation of Detection Baselines. To evaluate the performance of GlitchProber, we compared it with two implemented benchmark schemes and a recent testing method, GlitchHunter.

(1) **Exhaustive Search:** Each token in the token list is individually fed into the model, which performs tasks such as paraphrasing, spelling, and length calculation for each token.
(2) **Rule-based Random Sampling:** First, randomly select half of the tokens from the language model to form a candidate set. Since common English words typically do not become glitch tokens, use the Natural Language Toolkit (NLTK) to remove high-frequency English words from the candidate set. The remaining tokens are considered potential glitch tokens.
(3) **GlitchHunter:** This is the state-of-the-art automated detection method for glitch tokens [21].

Evaluation Metrics of Detection. For efficiency evaluation, we consider the **Time Cost** required to process all glitch tokens in the complete token list of a model. For GlitchProber, it encompasses the total duration including feature extraction, classifier training, identification and validation. For effectiveness evaluation, we consider **True Positive**, **Precision**, **Recall** and **F1-Score**.

Evaluation Settings. In our detection experiments, we evaluated the performance of GlitchProber with SVM regularization parameter and degree[33, 37] set to $C = 1, degree = 3$. We employ $\gamma = 0.1$ for random sampling and principal components to $P = 75$

Table 1: Summary of models in evaluation

Model Name	Number of Parameters	Vocabulary Size	Hidden Layers	Intermediate Size	Attention Heads
Llama-2-7b-chat	6.74B	32,000	32	11,008	32
Mistral-7B-Instruct-v0.1	7.24B	32,000	32	14,336	32
Qwen-7B-Chat	7.72B	151,936	32	22,016	32
Gemma-2b-it	2.51B	256,000	18	16,384	8
Yi-6B-Chat	6.06B	64,000	32	11,008	32

Table 2: Time cost comparison of GlitchProber and other baselines on different LLMs.

Test Model	Exhaustive Search	GlitchHunter	GlitchProber (ours)
Llama-2-7b-chat	619min 43s	74min 11s	**61min 38s**
Mistral-7B-Instruct-v0.1	651min 17s	64min 26s	**42min 39s**
Qwen-7B-Chat	2,228min 23s	720min 42s	**92min 48s**
Gemma-2b-it	3,575min 9s	681min 16s	**96min 43s**
Yi-6B-Chat	974min 4s	825min 25s	**140min 57s**
Average Time Cost	1,609min 42s	473min 11s	**89min 9s**

for PCA. For the rule-based random sampling methods, we conducted 100 independent experiments and averaged the results to obtain statistically significant conclusions. For the GlitchHunter method, we used the default settings from the original paper [21].

5.2 RQ3 (Efficient Detection): How efficient is our approach in identifying glitch tokens across different LLMs?

To evaluate the efficiency of GlitchProber, time overhead and the accuracy comparison results of various methods on five large models are shown in Table 2 and Table 3.

Our experimental results demonstrate that GlitchProber reached a detection efficiency advantage. Meanwhile, GlitchProber achieves a 100% precision which matches the performance of both GlitchHunter and the exhaustive search benchmark method. This signifies GlitchProber's minimal false positive rate. Furthermore, GlitchProber achieves a recall rate of 64.47%, surpassing GlitchHunter's 26.52%. The F1-score indicates that GlitchProber strikes a fine balance between precision and recall, efficiently detecting glitch tokens while maintaining high accuracy.

Answer to RQ3

GlitchProber achieves perfect accuracy across all test cases with low time overhead, exhibiting superior stability and performance in glitch token detection. Its efficiency gains are primarily attributed to its strategic adoption of small-scale sampling and intermediate layer feature extraction techniques, significantly enhancing detection efficacy.

5.3 RQ4 (Ablation Study): How do the different components of GlitchProber affect the detection results?

To assess the importance of the components in GlitchProber, we performed an ablation study across five models. We developed two variants: GlitchProber-No-PCA and GlitchProber-No-Post. GlitchProber-No-PCA omits the PCA during feature processing, while GlitchProber-No-Post eliminates the final token validation steps in the original GlitchProber. The comprehensive results are shown in Table 4.

Table 3: Performance comparison of GlitchProber and other baselines on different LLMs

Test Model	Metric	Rule-based Random Sampling	GlitchHunter	GlitchProber
Llama-2-7b-chat	TP	2,936	1,955	4,446
	Precision	24.74%	100.00%	100.00%
	Recall	45.70%	30.43%	69.22%
	F1-score	0.3210	0.4724	0.8181
Mistral-7B-Instruct-v0.1	TP	1,288	1,233	1,873
	Precision	11.44%	100.00%	100.00%
	Recall	46.35%	44.37%	67.41%
	F1-score	0.1836	0.6147	0.8053
Qwen-7B-Chat	TP	15,419	4,031	19,366
	Precision	21.04%	100.00%	100.00%
	Recall	50.24%	14.42%	63.08%
	F1-score	0.2966	0.2521	0.7736
Gemma-2b-it	TP	13,777	3,240	17,387
	Precision	11.30%	100.00%	100.00%
	Recall	49.27%	10.56%	62.18%
	F1-score	0.1838	0.1910	0.7668
Yi-6B-Chat	TP	3,215	2,662	4,900
	Precision	13.10%	100.00%	100.00%
	Recall	39.67%	32.84%	60.45%
	F1-score	0.1969	0.4944	0.7535
Average Performance	Precision	16.32%	100.00%	100.00%
	Recall	46.24%	26.52%	64.47%
	F1-score	0.2364	0.4049	0.7835

Table 4: Comparison of performance and memory usage for GlitchProber with different feature configurations across various language models.

Model	Metrics	GlitchProber	GlitchProber-No-Post	GlitchProber-No-PCA
Llama-2-7b-chat	F1-Score	0.8529	0.6097	–
	Memory	103.71GB	101.22GB	250.00GB
Mistral-7B-Instruct-v0.1	F1-Score	0.8652	0.5429	–
	Memory	107.11GB	102.67GB	250.00GB
Qwen-7B-Chat	F1-Score	0.8854	0.5510	–
	Memory	109.03GB	101.20GB	250.00GB
gemma-2b-it	F1-Score	0.8143	0.4226	–
	Memory	131.04GB	127.96GB	250.00GB
Yi-6B-Chat	F1-Score	0.8718	0.4507	–
	Memory	83.32GB	82.76GB	250.00GB

Note: '–' denotes incomplete experiment due to exceeding the maximum memory of our server of 250.00GB.

Table 4 offers a detailed comparative analysis of GlitchProber alongside its variants. With respect to the F1-score, GlitchProber markedly surpasses GlitchProber-No-Post, demonstrating average improvements of 0.32. These findings underscore the vital importance of implementing robust post-classification enhancements to boost overall performance. Furthermore, the absence of PCA in GlitchProber-No-PCA leads to substantial increases in memory usage beyond the maximum capacity of our server, resulting in the incompletion of GlitchProber-No-PCA. This starkly highlights the necessity of dimensionality reduction as a means to optimize resource allocation and ensure system stability.

Answer to RQ4

The PCA dimensionality reduction and the post-process are crucial components for GlitchProber. Omitting either of these components leads to a significant decrease in effectiveness, undermining the utility of GlitchProber.

6 EVALUATION OF GLITCH TOKEN FIX

6.1 Experiment setup

Evaluation Baselines of Fix. Due to the lack of existing methods for fixing glitch tokens, we compared GlitchProber with a benchmark scheme: a rule-based fix method, to verify its effectiveness. Specifically, the rule-based fix method does not rely on specific activation value differences to determine α and β. Instead, it directly